

Introdução ao **HTML e CSS**

Estrutura e estilo da web — os blocos fundamentais de qualquer página e aplicação web.

HTML 5

Estrutura

CSS 3

Estilo

Front-end

Foco da Aula

Professor: **Jackson Meires**

O que é HTML?

HTML (*HyperText Markup Language*) é a linguagem de **marcação** usada para estruturar o conteúdo de páginas web. Não é uma linguagem de programação — é uma linguagem de descrição de estrutura.



Estrutura

Define o que é um título, parágrafo, lista, tabela, formulário, link...



Tags (Marcadores)

Todo elemento é definido por uma tag: `<h1>`, `<p>`, `<div>`...



Padrão W3C

Mantido pelo W3C (World Wide Web Consortium). Versão atual: HTML5.



Base da Web

Todo site, toda aplicação web começa com HTML. CSS e JS complementam.

Analogia: HTML é o *esqueleto* de uma página. CSS é a *pele e roupa*. JavaScript é o *movimento*.

Estrutura Básica de um Documento HTML

INDEX.HTML

```
<!DOCTYPE html>
<html lang="pt-br">

  <head>
    <meta charset="UTF-8">
    <meta name="viewport"
      content="width=device-width, initial-scale=1.0">
    <title>Minha Página</title>
    <link rel="stylesheet" href="style.css">
  </head>

  <body>
    <h1>Olá, Mundo!</h1>
    <p>Meu primeiro parágrafo.</p>
    <script src="app.js"></script>
  </body>

</html>
```

Elemento	Função
<!DOCTYPE html>	Declara que é HTML5
<html>	Raiz do documento
<head>	Metadados (invisíveis ao usuário)
<meta charset>	Codificação UTF-8 (suporta acentos)
<meta viewport>	Responsividade mobile
<title>	Título na aba do browser
<body>	Conteúdo visível da página

Dica: No VS Code, digite ! e pressione Tab em um arquivo .html para gerar essa estrutura automaticamente (Emmet).

Tags de Texto e Títulos

TEXTO.HTML

```
<!-- Títulos: h1 ao h6 -->
<h1>Título Principal</h1>
<h2>Subtítulo</h2>
<h3>Seção</h3>

<!-- Parágrafos -->
<p>Este é um parágrafo de texto.</p>

<!-- Ênfase -->
<strong>Texto em negrito</strong>
<em>Texto em itálico</em>

<!-- Quebra de linha -->
<br>

<!-- Linha horizontal -->
<hr>

<!-- Código inline -->
<code>echo "Olá";</code>

<!-- Span: container inline -->
Cor: <span style="color:red">vermelho</span>
```

Resultado no browser:

Título Principal

Subtítulo

Este é um parágrafo de texto.

Texto em negrito e *itálico*

Código: `echo "Olá";`

Cor: **vermelho**

h1 por página: Use apenas um <h1> por página — é importante para SEO e acessibilidade.

Links e Imagens

LINKS_IMAGENS.HTML

```
<!-- Link básico -->
<a href="https://ifsc.edu.br">
  Site do IFSC
</a>

<!-- Link em nova aba -->
<a href="https://php.net"
  target="_blank"
  rel="noopener noreferrer">
  PHP.net
</a>

<!-- Link para seção interna -->
<a href="#contato">Ir ao contato</a>
<section id="contato">...</section>

<!-- Imagem -->


<!-- Imagem como link -->
<a href="pagina.html">
```

Atributos importantes do <a>:

Atributo	Valores
href	URL, #id, mailto:, tel:
target	_blank (nova aba), _self (mesma)
rel	noopener noreferrer (segurança)

Atributos importantes do :

Atributo	Função
src	Caminho ou URL da imagem
alt	Texto alternativo (acessibilidade, SEO)
width / height	Dimensões em px ou %

O atributo **alt** é obrigatório por acessibilidade. Descreva o conteúdo da imagem.

Listas

LISTAS.HTML

```
<!-- Lista não ordenada (ul) -->
```

```
<ul>
```

```
  <li>HTML</li>
```

```
  <li>CSS</li>
```

```
  <li>JavaScript</li>
```

```
</ul>
```

```
<!-- Lista ordenada (ol) -->
```

```
<ol>
```

```
  <li>Instalar o Laragon</li>
```

```
  <li>Criar o arquivo PHP</li>
```

```
  <li>Testar no browser</li>
```

```
</ol>
```

```
<!-- Lista aninhada -->
```

```
<ul>
```

```
  <li>Front-end
```

```
    <ul>
```

```
      <li>HTML</li>
```

```
      <li>CSS</li>
```

```
    </ul>
```

```
  </li>
```

```
  <li>Back-end
```

```
    <ul>
```

```
      <li>PHP</li>
```

```
    </ul>
```

Resultado:

Lista não ordenada:

- HTML
- CSS
- JavaScript

Lista ordenada:

1. Instalar o Laragon
2. Criar o arquivo PHP
3. Testar no browser

Lista aninhada:

- Front-end
 - HTML
 - CSS
- Back-end
 - PHP

Tabelas

TABELA.HTML

```
<table>
  <!-- Cabeçalho -->
  <thead>
    <tr>
      <th>Nome</th>
      <th>Serviço</th>
      <th>Preço</th>
    </tr>
  </thead>

  <!-- Corpo -->
  <tbody>
    <tr>
      <td>Ana</td>
      <td>Hidráulica</td>
      <td>R$ 150</td>
    </tr>
    <tr>
      <td>Bob</td>
      <td>Elétrica</td>
      <td>R$ 200</td>
    </tr>
  </tbody>
</table>
```

Resultado:

Nome	Serviço	Preço
Ana	Hidráulica	R\$ 150
Bob	Elétrica	R\$ 200

Elementos da tabela:

Tag	Função
<table>	Container principal
<thead>	Grupo do cabeçalho
<tbody>	Grupo do corpo
<tr>	Linha (<i>table row</i>)
<th>	Célula de cabeçalho
<td>	Célula de dados

Formulários

FORMULARIO.HTML

```
<form action="processar.php" method="POST">

  <label for="nome">Nome:</label>
  <input type="text" id="nome"
        name="nome" required>

  <label for="email">Email:</label>
  <input type="email" id="email"
        name="email">

  <label for="preco">Preço:</label>
  <input type="number" id="preco"
        name="preco" min="0" step="0.01">

  <label for="cat">Categoria:</label>
  <select id="cat" name="categoria">
    <option value="hidraulica">Hidráulica</option>
    <option value="elettrica">Elétrica</option>
  </select>

  <textarea name="descricao" rows="3"></textarea>

  <button type="submit">Enviar</button>
</form>
```

Tipos de <input>:

type	Uso
text	Texto livre
email	Valida formato de e-mail
password	Oculto o texto digitado
number	Só aceita números
date	Seletor de data
checkbox	Caixa marcável
radio	Opção única do grupo
file	Upload de arquivo
hidden	Campo invisível

O atributo **name** define o nome da variável recebida pelo PHP via `$_POST[ 'name' ]`.

HTML Semântico

O HTML5 introduziu tags **semânticas** que descrevem o significado do conteúdo — não apenas o visual. Melhoram SEO, acessibilidade e legibilidade.

SEMANTICO.HTML

```
<header>
  <nav>
    <a href="/">Início</a>
    <a href="/servicos">Serviços</a>
  </nav>
</header>

<main>
  <section>
    <h2>Serviços Disponíveis</h2>
    <article>
      <h3>Hidráulica</h3>
      <p>Reparos em tubulações.</p>
    </article>
  </section>

  <aside>
    <p>Dica: solicite orçamento!</p>
  </aside>
</main>

<footer>
```



<header>

Cabeçalho da página ou seção.
Geralmente contém logo e navegação.



<nav>

Links de navegação principal do site.



<main>

Conteúdo principal único da página.



<section>

Agrupamento temático de conteúdo.



<article>

Conteúdo independente (post, card, produto).



<footer>

Rodapé da página — copyright, links adicionais.

O que é CSS?

CSS (*Cascading Style Sheets*) é a linguagem de **estilo** usada para definir aparência: cores, fontes, tamanhos, espaçamentos, layouts e animações.

3 formas de aplicar CSS:

1. EXTERNO (RECOMENDADO)

```
<!-- no <head> do HTML -->
<link rel="stylesheet" href="style.css">
```

2. INTERNO (NO <HEAD>)

```
<style>
  h1 { color: blue; }
</style>
```

3. INLINE (EVITAR)

```
<h1 style="color: blue;">Título</h1>
```

Anatomia de uma regra CSS:

```
/* Seletor { propriedade: valor; } */
```

```
h1 {
  color: #333;
  font-size: 2rem;
  font-weight: bold;
  margin-bottom: 16px;
}
```

```
p {
  color: #666;
  line-height: 1.6;
}
```

```
/* Múltiplos seletores */
h1, h2, h3 {
  font-family: 'Segoe UI', sans-serif;
}
```

Cascata: quando há conflito, vence a regra mais específica ou a última

Seletores CSS

SELETORES.CSS

```
/* Elemento */
p { color: gray; }

/* Classe (.) – reutilizável */
.destaque { color: orange; font-weight: bold; }
.card      { border: 1px solid #ccc; padding: 16px; }

/* ID (#) – único por página */
#cabecalho { background: #1e293b; }

/* Filho direto (>) */
ul > li { list-style: none; }

/* Descendente (espaço) */
.card p { font-size: .9rem; }

/* Pseudo-classe */
a:hover { color: blue; text-decoration: underline; }
tr:nth-child(even) { background: #f8fafc; }
input:focus { outline: 2px solid blue; }

/* Atributo */
input[type="email"] { border-color: green; }
```

Especificidade (ordem de prioridade):

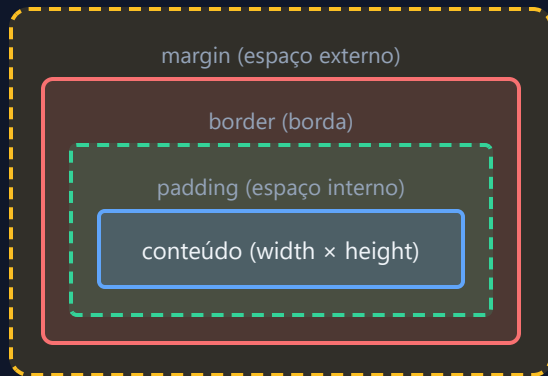
Seletor	Peso
Elemento (p)	0,0,1
Classe (.card)	0,1,0
ID (#header)	1,0,0
Inline (style="")	1,0,0,0
!important	Sobrescreve tudo

Prefira **classes** sobre IDs para estilização. IDs são para JavaScript e âncoras.

Evite !important — use-o apenas em último recurso, pois dificulta a manutenção.

Box Model — O Modelo de Caixa

Todo elemento HTML é uma **caixa retangular** composta por 4 camadas:



BOX-MODEL.CSS

```
.card {  
  /* Conteúdo */  
  width: 300px;  
  height: 200px;  
  
  /* Espaço interno */  
  padding: 16px;           /* todos os lados */  
  padding: 8px 16px;      /* vertical horizontal */  
  padding-top: 8px;  
  
  /* Borda */  
  border: 1px solid #ccc;  
  border-radius: 8px;      /* cantos arredondados */  
  
  /* Espaço externo */  
  margin: 16px;  
  margin: 0 auto;         /* centralizar */  
  
  /* Inclui padding e border no width */  
  box-sizing: border-box;  
}
```

Sempre use `box-sizing: border-box` — evita cálculos de tamanho. Coloque

Flexbox — Layout Flexível

FLEXBOX.CSS

```
/* Container flex */
.container {
  display: flex;

  /* Direção dos itens */
  flex-direction: row;      /* → (padrão) */
  flex-direction: column;   /* ↓ */

  /* Alinhamento horizontal */
  justify-content: flex-start; /* início */
  justify-content: center;     /* centro */
  justify-content: space-between; /* espaço entre */
  justify-content: space-around; /* espaço ao redor */

  /* Alinhamento vertical */
  align-items: center;
  align-items: stretch;      /* preencher (padrão) */

  /* Quebra de linha */
  flex-wrap: wrap;
  gap: 16px;                  /* espaço entre itens */
}
```

```
/* Item flex */
```

```
.item {
```

Casos de uso comuns:

```
/* Centralizar vertical e horizontal */
.centralizado {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}
```

```
/* Navbar */
nav {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0 24px;
}
```

```
/* Cards em linha */
.cards {
  display: flex;
  flex-wrap: wrap;
  gap: 16px;
}
```

Flexbox é ideal para layouts **unidimensionais** (linha ou coluna). Para layouts 2D

CSS Grid — Layout em Grade

GRID.CSS

```
/* Container Grid */
.grid {
  display: grid;

  /* 3 colunas iguais */
  grid-template-columns: 1fr 1fr 1fr;

  /* Repeat + auto-fit (responsivo) */
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));

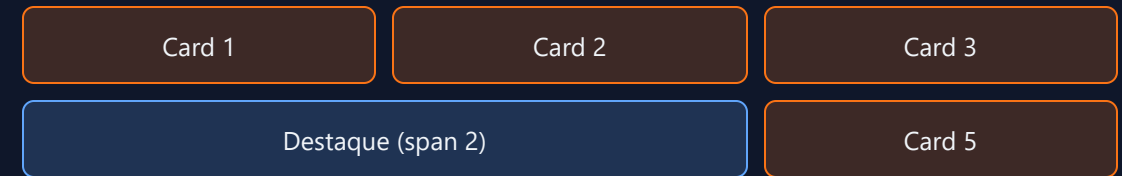
  /* Colunas de tamanhos diferentes */
  grid-template-columns: 250px 1fr; /* sidebar + main */

  gap: 16px; /* espaço entre células */
}

/* Item ocupando múltiplas colunas */
.destaque {
  grid-column: 1 / 3; /* da col 1 até 3 */
  grid-column: span 2; /* ocupar 2 colunas */
}

/* Layout clássico de página */
.pagina {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
}
```

Exemplo visual — grid de cards:



fr = fração do espaço disponível. 1fr 2fr = proporção 1:2 entre as colunas.

`repeat(auto-fit, minmax(200px, 1fr))` cria um grid responsivo automaticamente — o número de colunas se ajusta ao espaço disponível.

Cores e Tipografia

CORES.CSS

```
/* Formatos de cor */
p {
  color: red;           /* nome */
  color: #1e293b;       /* hexadecimal */
  color: rgb(30, 41, 59); /* rgb */
  color: rgba(0, 0, 0, .5); /* rgba (opacidade) */
  color: hsl(220, 40%, 18%); /* hsl */
}

/* Variáveis CSS (custom properties) */
:root {
  --cor-primaria: #f97316;
  --cor-texto: #1e293b;
}
h1 { color: var(--cor-primaria); }

/* Gradiente */
.hero {
  background: linear-gradient(135deg, #f97316, #fbbf24);
}
```

TIPOGRAFIA.CSS

```
body {
  font-family: 'Segoe UI', system-ui, sans-serif;
  font-size: 16px; /* base (1rem = 16px) */
  line-height: 1.6;
}

h1 {
  font-size: 2.5rem; /* 40px */
  font-weight: 700; /* bold */
  letter-spacing: -.5px;
}

/* Google Fonts */
<link href="https://fonts.googleapis.com/css2?
  family=Inter:wght@400;700&display=swap"
  rel="stylesheet">

body { font-family: 'Inter', sans-serif; }

/* Unidades de texto */
/* px = fixo | rem = relativo à raiz | em = relativo ao pai */
```

Responsividade e Media Queries

RESPONSIVO.CSS

```
/* Mobile First: estilos base para mobile */
.cards {
  display: grid;
  grid-template-columns: 1fr; /* 1 coluna */
  gap: 16px;
}

/* Tablet (≥ 768px) */
@media (min-width: 768px) {
  .cards {
    grid-template-columns: repeat(2, 1fr);
  }
}

/* Desktop (≥ 1024px) */
@media (min-width: 1024px) {
  .cards {
    grid-template-columns: repeat(3, 1fr);
  }
  nav {
    display: flex;
  }
}

/* Ocultar em mobile */
@media (max-width: 767px) {
```

Breakpoints comuns:

Nome	Largura
Mobile	< 768px
Tablet	768px – 1023px
Desktop	≥ 1024px
Wide	≥ 1280px

Mobile First: comece com estilos para telas pequenas e use min-width para adicionar estilos progressivos. É a abordagem recomendada.

Não esqueça a meta tag viewport no HTML: `<meta name="viewport" content="width=device-width, initial-scale=1.0">`

HTML + CSS + PHP — Juntos na Prática

SERVICOS.PHP — HTML DINÂMICO COM PHP

```
<?php
// Dados do banco de dados
$servicos = [
    ['titulo' => 'Hidráulica', 'preco' => 150.00],
    ['titulo' => 'Elétrica', 'preco' => 200.00],
];
?>
<!DOCTYPE html>
<html lang="pt-br">
<head>
    <meta charset="UTF-8">
    <link rel="stylesheet" href="style.css">
    <title>Serviços</title>
</head>
<body>
    <main class="container">
        <h1>Serviços Disponíveis</h1>
        <div class="cards">
            <?php foreach ($servicos as $s): ?>
                <div class="card">
                    <h2><?= htmlspecialchars($s['titulo']) ?></h2>
                    <p>R$ <?= number_format($s['preco'], 2, ',', '.') ?></p>
                </div>
            <?php endforeach; ?>
        </div>
    </main>
</body>
</html>
```

STYLE.CSS

```
* { box-sizing: border-box; margin: 0; padding: 0; }

body {
    font-family: 'Segoe UI', sans-serif;
    background: #f1f5f9;
}

.container {
    max-width: 1000px;
    margin: 0 auto;
    padding: 32px 16px;
}

.cards {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
    gap: 16px;
}

.card {
    background: white;
    border-radius: 10px;
    padding: 20px;
    box-shadow: 0 2px 8px rgba(0,0,0,.08);
}
```

Exercício Prático

Crie uma página HTML com CSS que sirva de **front-end** para o sistema de serviços:

1. Crie `index.html` com a estrutura semântica completa (`header`, `main`, `footer`).
2. No header, adicione uma **navbar** com o nome do app e links "Início" e "Serviços".
3. No main, adicione um **formulário** com os campos: título, descrição, preço e categoria.
4. Abaixo do formulário, crie uma **tabela** (estática por enquanto) com 3 serviços de exemplo.
5. Crie `style.css` separado e aplique: cores, fontes, espaçamentos e layout com Flexbox ou Grid.
6. **Bônus:** torne o layout responsivo com Media Queries.
7. **Bônus 2:** converta o HTML estático para um arquivo `.php` que gere os dados dinâmicos.

Dica: Inspecione elementos no browser com F12 (DevTools) — visualize o Box Model, edite CSS em tempo real e veja erros no console.

Ferramentas Úteis



VS Code + Extensões

Live Server — recarrega o browser ao salvar. **Prettier** — formata o código automaticamente.



DevTools (F12)

Inspect, Console, Network, Sources. Indispensável para debug de HTML, CSS e requisições HTTP.



Color Hunt

colorhunt.co — paletas de cores prontas e harmoniosas para seus projetos.



Flexbox Froggy

flexboxfroggy.com — jogo interativo para aprender Flexbox de forma divertida.



CSS Grid Garden

cssgridgarden.com — jogo para praticar CSS Grid.



MDN Web Docs

developer.mozilla.org — documentação de referência HTML e CSS (em pt-BR).

Resumo — O que aprendemos



Estrutura HTML

DOCTYPE, html, head, body — e as tags semânticas do HTML5.



Tags Essenciais

Títulos, parágrafos, links, imagens, listas, tabelas e formulários.



CSS Básico

Seletores, especificidade, propriedades de texto, cores e background.



Box Model

margin, border, padding, content — e box-sizing: border-box.



Flexbox e Grid

Layouts modernos, responsivos e sem hacks de float.



Responsividade

Media queries, Mobile First e viewport meta tag.

Próximo tema: PHP — adicionar lógica, processar formulários e conectar ao banco de dados!